

Air University Aerospace And Aviation Campus Kamra

Department of Computer Science



MyShell A Custom Unix Shell

Project Members:

- Haleema Abid(245053)
- Samreen Shoukat(245077)
- Hamna Saleem(245047)

Semester: BSCYS-4B

Submitted To: Mam Kashaf

Date: May 15, 2026

Contents

MyShell - A Custom Unix Shell	3
1. Introduction.....	3
2. Objectives	3
3. Features Implemented.....	3
3.1.Colorized Prompt with Timestamp	3
3.2.Welcome Banner	4
3.3.Command History.....	4
3.4.Built-in Commands	4
3.5.I/O Redirection	4
3.6.Piping.....	4
3.7.Background Execution.....	4
4. System Calls Used.....	5
5. Code Structure	5
6. How to Compile & Run	6
7. Ouput & Verification Of Commands	7
7.1.Banner & Prompt.....	7
7.2.help command	7
7.3.cd command.....	7
7.4. History command.....	8
7.5.Repeat(!) command.....	8
7.6. Output Redirection	8
7.7.Append Redirection.....	8
7.8.Input Redirection	8
7.9.Pipe.....	9
7.10.Background(&).....	9
7.11.Exit/Quit	9
8. Conclusion	9

MyShell - A Custom Unix Shell

1. Introduction

A shell is a command-line interface that allows users to interact with the operating system by typing commands. It acts as a bridge between the user and the kernel, interpreting commands and executing them. Shells are fundamental components of any Unix/Linux system and provide mechanisms such as process creation, I/O redirection, piping, and job control.

MyShell is a custom Unix-like shell developed in C as part of our Operating Systems course project. It simulates the core functionality of standard shells like bash or sh, including command execution, I/O redirection, piping between commands, background execution, and a built-in command history. MyShell also features a colorized interface with a live timestamp prompt, making it both functional and visually informative.

2. Objectives

The primary objectives of this project were:

- To understand how a Unix shell works internally at the system call level
- To implement core shell features including command parsing, process forking, and execution
- To support input/output redirection (>, >>, <) and piping (|)
- To implement background process execution using the & operator
- To maintain a command history and support the !! repeat mechanism
- To build a user-friendly, colorized shell interface with a live timestamped prompt

3. Features Implemented

3.1. Colorized Prompt with Timestamp

MyShell displays a dynamic prompt that shows the current system time (HH:MM:SS) and the current working directory. ANSI escape codes are used to color the prompt in green and cyan, making it easy to distinguish from command output.

3.2>Welcome Banner

On startup, MyShell displays an ASCII art banner with a welcome message personalized with the current username, retrieved using `getenv("USER")`.

3.3.Command History

MyShell maintains a circular history of the last 10 commands entered by the user. Users can view the full history using the `history` command. The `!!` command repeats the most recently executed command.

3.4.Built-in Commands

The following built-in commands are natively supported:

- `cd <dir>` — Changes the current working directory
- `history` — Displays the list of previously entered commands
- `!!` — Repeats the last command
- `help` — Displays a help menu listing all supported commands
- `exit / quit` — Exits the shell gracefully

3.5.I/O Redirection

MyShell supports the following redirection operators:

- `> file` — Redirects standard output to a file (overwrite mode)
- `>> file` — Redirects standard output to a file (append mode)
- `< file` — Redirects standard input from a file

3.6.Piping

The shell supports single-level piping using the `|` operator. The output of the left command is connected to the input of the right command via a kernel pipe, using `pipe()`, `fork()`, and `dup2()` system calls.

3.7.Background Execution

Commands suffixed with `&` are executed in the background. The shell prints the background process's PID and immediately returns to the prompt without waiting for the child to finish.

4. System Calls Used

MyShell relies on the following POSIX system calls:

System Call	Purpose
fork()	Creates a child process to execute a command
execvp()	Replaces child process image with the new command
waitpid()	Parent waits for a specific child process to finish
pipe()	Creates a unidirectional pipe between two processes
dup2()	Redirects file descriptors for I/O redirection and piping
open()	Opens files for reading or writing during redirection
close()	Closes file descriptors after use to prevent leaks
getcwd()	Retrieves the current working directory for the prompt
chdir()	Changes the current working directory (used by cd)
getenv()	Reads environment variables like USER and HOME

5. Code Structure

The project is contained in a single C file (myshell.c) organized into the following functional modules:

Function	Description
show_banner()	Displays the ASCII art welcome banner on startup
get_prompt()	Returns the formatted prompt string with time and cwd
save_to_history()	Saves a command to the circular history buffer
show_history()	Prints all saved commands with numbered indices
get_last_command()	Returns the most recently saved command
strip_newline()	Removes trailing newline from input string

parse_args()	Tokenizes input into an argument array; detects &
cmd_cd()	Implements the cd built-in command
cmd_help()	Prints the formatted help menu
cmd_exit()	Sets shell_running flag to 0, exits shell
setup_redirection()	Sets up file descriptors for I/O redirection
run_piped_commands()	Forks two processes connected via a pipe
execute_command()	Main dispatcher: built-ins, pipes, or fork+exec
main()	Shell loop: prompt → input → parse → execute

6. How to Compile & Run

MyShell requires a standard GCC compiler on a Linux/Unix system. To compile and run:

```
# Compile the shell
gcc myshell.c -o myshell

# Run the shell
./myshell
```

Example commands to test:

- `ls -la` (basic command)
- `ls | grep .c` (piping)
- `ls > output.txt` (output redirection)
- `cat < output.txt` (input redirection)
- `sleep 5 &` (background execution)
- `history` (view command history)
- `!!` (repeat last command)

7. Output & Verification Of Commands

7.1.Banner & Prompt

[illegible]

7.4. History command

```
11:56:12:/home/haleema/Desktop> history
— Command History —
[1] help
[2] cd
[3] cd Desktop
[4] cd /nonexistent
[5] history
11:57:18:/home/haleema/Desktop> 
```

7.5.Repeat(!!) command

```
11:57:18:/home/haleema/Desktop> !!
Repeating: history
— Command History —
[1] help
[2] cd
[3] cd Desktop
[4] cd /nonexistent
[5] history
11:58:09:/home/haleema/Desktop> 
```

7.6. Output Redirection

```
11:58:09:/home/haleema/Desktop> echo hello > out.txt
12:01:06:/home/haleema/Desktop> 
```



7.7.Append Redirection

```
12:01:06:/home/haleema/Desktop> echo "this is my shell" >> out.txt
12:03:55:/home/haleema/Desktop> 
```



7.8.Input Redirection

```
12:06:00:/home/haleema/Desktop> cat < out.txt
hello
"this is my shell"
12:06:15:/home/haleema/Desktop> 
```


7.9.Pipe

```
12:06:15:/home/haleema/Desktop> ls | grep .txt
out.txt
12:07:28:/home/haleema/Desktop> 
```

7.10.Background(&)

```
12:09:06:/home/haleema/Desktop> sleep 5 &
[Background PID: 4573]
12:09:51:/home/haleema/Desktop> ls
out.txt  shell  shell.c
12:09:52:/home/haleema/Desktop> 
```

7.11.Exit/Quit

```
12:09:52:/home/haleema/Desktop> quit

Goodbye! Exiting MyShell...
haleema@haleema-VMware-Virtual-Platform:~/Desktop$ 
```

8. Conclusion

MyShell successfully demonstrates the core principles of operating system interaction at the shell level. By implementing process creation with `fork()` and `exec()`, inter-process communication through pipes, and file descriptor manipulation for redirection, this project provided hands-on exposure to the internals of how a Unix shell operates.

The project deepened our understanding of process management, system calls, and the POSIX API. It also highlighted the complexity involved in correctly handling edge cases such as argument parsing, background processes, and error handling. MyShell serves as a solid foundation that could be extended with features like tab completion, multiple pipes, or signal handling in future iterations.